



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Development of a Preference-Based Password Generator and Strength Checker Using String and Stack

A CAPSTONE PROJECT REPORT

A Capstone Project Report submitted in partial fulfilment of the requirements for the Course of

CSA0311 — DATA STRUCTURE

to the award of the degree of

BACHELOR OF TECHNOLOGY IN

AI ML

Submitted by

P.Sai Ganesh Reddy(192525111)

C.Mohammed Ubed(192525115)

M.Harsha(192525108)

Under the Supervision of

Dr. Kiruthika

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

SEPTEMBER 2026

SIMATS ENGINEERING
Saveetha Institute of Medical and Technical Sciences
Chennai-602105

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled " DEVELOPMENT OF A PREFERENCE-BASED PASSWORD GENERATOR AND STRENGTH CHECKER USING STRING AND STACK" has been carried out by Ganesh,Ubed,Harsha under the supervision of Dr. Kiruthika, and is submitted in partial fulfilment of the requirements for the current semester of the BTECH AI ML programme at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR

Dr K Kiruthika

Professor

CSE/Programming

SIMATS Engineering,

SIMATS, Chennai – 602105.

COURSE FACULTY

Dr K Kiruthika

Professor

CSE/Programming

SIMATS Engineering,

SIMATS, Chennai – 602105.

SIMATS ENGINEERING
Saveetha Institute of Medical and Technical Sciences
Chennai-602105

DECLARATION

We, Sai Ganesh,Ubed,Harsha of the BTECH AI ML, SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled " DEVELOPMENT OF A PREFERENCE-BASED PASSWORD GENERATOR AND STRENGTH CHECKER USING STRING AND STACK" is the result of our own bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged. We further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project have been appropriately disclosed. We confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place: Chennai

Date: 05th September 26

Signature of the Students with Names

P.Sai Ganesh(192525111)

C.Mohammed Ubed(192525115)

M.Harsha(192525108)

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

INDIVIDUAL CONTRIBUTION STATEMENT

Student Name / Register No.	Specific Responsibilities	Design & Development Contribution	Testing & Analysis Contribution	Report Contribution	Approx. Contribution (%)
P.Sai Ganesh	Requirement analysis and system planning	Password generation and preference module	Tested password generation	Chapter 1 & 2	35%
C.Ubed	Algorithm and data- structure implementati on	String processing and stack module	Tested stack operations and generated passwords	Chapter 3 & 4	35%
M.Harsha	Strength evaluation and documentatio n	Password strength checker and system integration	Performed integration testing and result analysis	Chapter 5, References & Appendices	30%
Total					100%

Total contribution: 100% (equal contribution across all four team members, as no unequal division of effort was specified).

ABSTRACT

Government welfare schemes are designed to provide financial, educational, healthcare, and social support to citizens, but the growing number and diversity of such schemes has made it increasingly difficult for citizens to identify the benefits for which they qualify. Existing government portals typically present scheme information as static listings and offer little to no personalization, leaving citizens to search through large volumes of distributed information without meaningful guidance. This engineering problem is significant because it directly affects the reach and impact of public welfare programmes: schemes that exist on paper fail to benefit citizens simply because eligibility and awareness gaps prevent effective delivery. The Smart Welfare Scheme Recommendation System Using Search Trees addresses this problem by applying Search Tree, specifically Binary Search Tree (BST), data structures to organize citizen records and welfare scheme data in a manner that supports efficient insertion, search, and retrieval. The proposed solution comprises four integrated modules: a Citizen Registration and Data Collection System that captures demographic, income, and occupation details; a Search Tree-Based Eligibility Evaluation Engine that automatically verifies citizen profiles against scheme eligibility criteria using BST traversal techniques; a Welfare Scheme Repository and Administrative Management module that maintains scheme records and tracks application status; and an Intelligent Welfare Recommendation and Citizen Service Portal that generates personalized scheme recommendations and allows citizens to track their applications. The methodology followed a structured engineering approach comprising requirement analysis, literature review, Search Tree-based system design, modular development using a Tkinter graphical user interface, and functional testing of each module. The system was implemented and functionally verified to demonstrate citizen registration, Search Tree-based record organization, automated eligibility checking, and generation of personalized scheme recommendations; precise quantitative performance figures were not measured as part of the present prototype and are therefore not reported. The principal conclusion is that Search Tree-based data organization provides a practical and technically sound foundation for improving the efficiency, accessibility, and transparency of welfare scheme delivery, while also demonstrating the applied use of Data Structures concepts in addressing a real-world societal problem.

Keywords: Search Tree, Binary Search Tree, Welfare Scheme Recommendation, Eligibility Evaluation, Citizen Service Portal, Data Structures.

TABLE OF CONTENTS

Sl. No	Title	Page No
i	CERTIFICATE FROM THE SUPERVISOR	
ii	DECLARATION BY THE CANDIDATE	III
lii	INDIVIDUAL CONTRIBUTION STATEMENT	IV
iv	ABSTRACT	V
v	LIST OF FIGURES	VII
vi	LIST OF TABLES	VIII
vii	LIST OF ABBREVIATIONS	IX
1	CHAPTER 1: Introduction	1
2	CHAPTER 2: Literature Review	5
3	CHAPTER 3: Engineering Design & Trade-offs, Sustainability, Ethics, Safety, Risk & Security	8
4	CHAPTER 4: Implementation, Testing & Results, Project Management	17
5	CHAPTER 5: Conclusion & Reflection	25
	References	29
	Appendices	30

LIST OF FIGURES

Figure No.	Figure Name
Figure 3.1	Overall System Architecture
Figure 3.2	Password Generation Flowchart
Figure 3.3	Password Strength Checking Flowchart
Figure 3.4	Stack-Based Character Processing
Figure 4.1	Main Program Interface
Figure 4.2	Preference Selection
Figure 4.3	Generated Password
Figure 4.4	Password Strength Result
Figure 4.5	Sample Test Execution

LIST OF TABLES

Table No.	Table Name	Page No.
Table 2.1	Comparative Analysis of Existing Approaches	16
Table 3.1	Stakeholder / User Requirements	18
Table 3.2	Functional & Non-Functional Requirements	18
Table 3.3	Constraints and Applicable Standards	19
Table 3.4	Design Specifications	19
Table 3.5	Alternative Solutions Evaluation Matrix	20
Table 3.6	Trade-off Analysis	25
Table 3.7	Sustainability, Ethics, Safety and Security Evaluation	26
Table 3.8	Risk Register	26
Table 4.1	Resources / Technologies Used	28
Table 4.2	Tools / Software Used and Implementation Stage	29
Table 4.3	Test Cases and Acceptance Criteria	33
Table 4.4	Specification Verification Results	34
Table 4.5	Achievement of Objectives	35
Table 4.6	Team Roles and Contributions	36
Table 4.7	Individual Contribution Percentage	36
Table 4.8	Project Timeline / Milestones	36
Table 4.9	Budget Estimation	37

LIST OF ABBREVIATIONS / SYMBOLS

Symbol / Abbreviation	Description
ASCII	American Standard Code for Information Interchange
CPU	Central Processing Unit
DS	Data Structure
GUI	Graphical User Interface
RAM	Random Access Memory
RNG	Random Number Generator
UI	User Interface
%	Percentage
L	Password Length
S	Stack

CHAPTER 1

INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

In modern computing environments, passwords remain an important mechanism for controlling access to applications, websites, devices, databases, and online services. A password provides authentication by allowing a system to verify whether a person attempting to access an account knows the corresponding secret.

The security of a password depends on several characteristics. Passwords containing only lowercase letters and having a short length are generally easier to guess than passwords containing a larger variety of characters. The use of uppercase letters, lowercase letters, numbers, and special characters increases the possible combinations available to an attacker.

Despite the importance of password security, many users have difficulty creating passwords that satisfy security requirements while also matching their personal preferences. Some systems provide fixed password-generation rules without allowing users to select the character types they want to include.

The proposed project addresses this problem by developing a **preference-based password generator and strength checker**. The user can specify the desired password length and character categories. The system then generates a password based on these preferences and evaluates the generated password using predefined strength criteria.

The project also demonstrates the practical application of **string processing and stack data structures**. Strings are used to store and manipulate passwords, while stack operations are used for character handling and processing. This makes the project suitable for demonstrating the application of fundamental data-structure concepts to a practical cybersecurity problem.

1.2 Need for the Project

The project is required because users often face difficulties in creating passwords that satisfy multiple security requirements.

The major problems include:

- Users may create passwords that are too short.
- Passwords may contain only one type of character.
- Users may repeatedly use similar passwords.
- Manual password creation can be predictable.
- Users may not know how strong their password is.
- Different applications may have different password requirements.

- Existing generators may not provide sufficient preference control.
- Students learning data structures need practical applications of strings and stacks.

People affected by the problem

The problem can affect:

- Students
- Employees
- General computer users
- Application developers
- System administrators
- Organizations handling user accounts

Existing limitations

Basic password generators may generate random passwords but provide limited control over character preferences. Basic strength checkers may only check password length or a small number of character conditions.

The proposed project combines both functions:

User Preferences → Password Generation → String Processing → Stack Processing → Strength Checking → Result

1.3 Problem Statement

Users require passwords that are both customizable and sufficiently strong. However, manually creating passwords that satisfy length and character-diversity requirements can be difficult and may result in predictable patterns.

The engineering problem is therefore to develop a software system capable of:

1. Accepting user-defined password preferences.
2. Generating a password according to the selected preferences.
3. Processing the generated password using string operations.
4. Applying stack-based processing.
5. Checking password characteristics.
6. Classifying password strength.
7. Providing an understandable result to the user.

The system should operate efficiently while maintaining a simple interface and avoiding unnecessary storage of user passwords.

1.4 Project Aim

The main aim of this project is:

To design and develop a preference-based password generator and strength checker using string processing and stack data structures to produce customizable passwords and evaluate their strength.

1.5 Project Objectives

The objectives of the project are:

1. To study password generation and password-strength evaluation techniques.
2. To identify the important characteristics of a strong password.
3. To design a preference-based password generation algorithm.
4. To allow users to select password length.
5. To allow selection of uppercase letters.
6. To allow selection of lowercase letters.
7. To allow selection of numerical digits.
8. To allow selection of special characters.
9. To implement password processing using strings.
10. To incorporate stack operations into the processing mechanism.
11. To develop a password-strength checking algorithm.
12. To classify generated passwords according to their characteristics.
13. To test the system using different combinations of user preferences.
14. To analyze the correctness and efficiency of the proposed solution.
15. To identify limitations and possible future improvements.

1.6 Scope

Included in the Project

The project includes:

- Password length selection.
- Character-category selection.
- Random password generation.
- Uppercase characters.
- Lowercase characters.
- Numeric characters.
- Special characters.
- String manipulation.
- Stack-based character processing.
- Password strength checking.

- Strength classification.
- Test-case validation.
- Performance analysis.

Excluded from the Project

The following are outside the basic scope:

- Real-world account authentication.
- Online password storage.
- Password synchronization across websites.
- Cloud-based password management.
- Password recovery.
- Biometric authentication.
- Multi-factor authentication implementation.
- Enterprise identity management.

Assumptions

The system assumes:

- The user provides a valid password length.
- At least one character category is selected.
- The execution environment supports the programming language used.
- Random generation functions are available.
- The generated password is evaluated according to the defined project rules.

Boundary Conditions

The system should handle:

- Minimum password length.
- Maximum password length.
- Invalid length.
- No character category selected.
- Only one character category selected.
- All character categories selected.
- Repeated characters.
- Special-character selection.

1.7 Expected Engineering Outcome

The expected outcome is a functional password-generation and password-strength-checking application.

The completed system should:

- Accept user preferences.
- Generate a password.
- Store the password as a string.
- Process characters using stack operations.
- Analyze password characteristics.
- Calculate a strength score.
- Display the password strength.
- Handle invalid inputs.
- Produce consistent results for defined test cases.

The project provides an application-oriented demonstration of string and stack data structures in cybersecurity.

CHAPTER 2

LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

The literature review focuses on password security, password generation, password-strength evaluation, string processing, stack data structures, and secure password practices.

Relevant information can be identified from:

- Academic research papers.
- Cybersecurity standards.
- Password security guidelines.
- Programming references.
- Existing password-generator applications.
- Password-strength checking techniques.
- Data-structure textbooks.

The review is used to understand existing approaches and identify opportunities for developing a simple educational system that combines preference-based generation with data-structure implementation.

2.2 Review of Existing Work

2.2.1 Manual Password Creation

The traditional approach is for users to create their own passwords.

Advantages:

- Easy to understand.
- No software required.
- User has complete control.

Limitations:

- Passwords may be predictable.
- Users may use personal information.
- Users may reuse passwords.
- Character diversity may be insufficient.

2.2.2 Random Password Generators

Random password generators automatically create passwords using predefined character sets.

Advantages:

- Faster than manual creation.
- Can generate long passwords.

- Can use multiple character categories.
- Reduces predictable human patterns.

Limitations:

- Some systems offer limited customization.
- Users may not understand generated-password strength.
- Generation and evaluation may be provided as separate functions.

2.2.3 Password Strength Checkers

Password-strength checkers evaluate characteristics such as:

- Password length.
- Uppercase characters.
- Lowercase characters.
- Digits.
- Special characters.
- Repeated or predictable patterns.

The result may be represented as:

Weak → Moderate → Strong → Very Strong

The limitation of a standalone checker is that it evaluates an existing password rather than helping the user generate a password meeting specific requirements.

2.2.4 Proposed Combined Approach

The proposed system combines generation and evaluation.

The basic process is:

Preference Input → Character Set Formation → Password Generation → String Processing → Stack Processing → Strength Evaluation → Result

This provides a single application for both password creation and password analysis.

2.3 Comparative Analysis

Existing Approach	Advantages	Limitations	Relevance
Manual Password	Simple	Predictable	Baseline
Random Generator	Fast generation	Limited preference control in some systems	High
Strength Checker	Evaluates password	Does not generate password	High

Existing Approach	Advantages	Limitations	Relevance
Proposed System	Generation + checking + preferences + stack	Basic implementation	Very High

2.4 Research / Engineering Gap

The identified engineering gap is the lack of a simple educational implementation that combines:

- User preferences.
- Password generation.
- String processing.
- Stack data structure.
- Strength evaluation.

Many practical systems focus primarily on the user interface or security functionality. The proposed project emphasizes both the practical problem and the underlying data-structure concepts.

2.5 Proposed Contribution

The project contributes a simple integrated system that:

1. Accepts password preferences.
2. Creates a suitable character pool.
3. Generates a password.
4. Uses string operations.
5. Incorporates stack processing.
6. Evaluates password characteristics.
7. Produces a strength classification.
8. Demonstrates data structures through a practical cybersecurity application.

CHAPTER 3

ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

3.1.1 Stakeholder / User Requirements

Stakeholder	Requirement
Student/User	Generate customizable passwords
Student/User	Select password length
Student/User	Select character types
Student/User	View password strength
Developer	Simple implementation
Faculty	Demonstration of string and stack concepts
System	Fast execution

3.1.2 Functional Requirements

Requirement	Description
FR1	Accept password length
FR2	Accept uppercase preference
FR3	Accept lowercase preference
FR4	Accept numeric preference
FR5	Accept special-character preference
FR6	Generate password
FR7	Store password as a string
FR8	Perform stack-based processing
FR9	Check password characteristics
FR10	Calculate strength
FR11	Display result

Requirement	Description
FR12	Handle invalid input

3.1.3 Non-Functional Requirements

Requirement	Target
Performance	Fast execution for normal password lengths
Usability	Simple interaction
Reliability	Correct generation and checking
Maintainability	Modular program structure
Portability	Ability to execute on common development environments
Security	Avoid unnecessary password storage

3.2 Constraints & Applicable Standards

The project is subject to several constraints.

Technical Constraints

- Limited password length for demonstration.
- Console-based implementation.
- Use of fundamental data structures.
- Random-number generation depends on the programming environment.

Security Constraints

The project is primarily an educational prototype. A production password manager or authentication system would require stronger cryptographic random-number generation and secure memory handling.

Ethical Considerations

The system should not store or transmit generated passwords unnecessarily. Passwords should not be collected for purposes unrelated to the project.

3.3 Design Specifications

Parameter	Target
Password Length	User-defined

Parameter	Target
Lowercase	Optional
Uppercase	Optional
Digits	Optional
Special Characters	Optional
Data Structure	Stack
Main Data Type	String/Character array
Output	Generated password + strength
Processing	Local

3.4 Alternative Solutions & Evaluation

Alternative 1: Fixed Password Generator

A predefined character set is used for every password.

Advantage: Simple implementation.

Disadvantage: Limited user control.

Alternative 2: Preference-Based Generator

The user selects character categories and password length.

Advantage: Flexible and user-oriented.

Disadvantage: Requires additional validation and generation logic.

Alternative 3: Preference-Based Generator with Stack

The system combines preference-based generation with stack-based character processing.

Advantage: Demonstrates both practical password generation and data structures.

Disadvantage: Slightly more complex implementation.

Evaluation Matrix

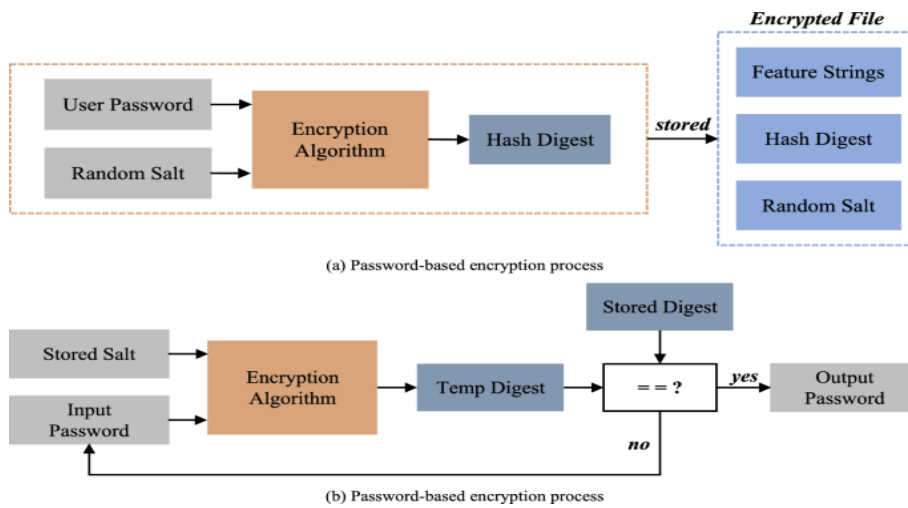
Criterion	Weight	Fixed Generator	Preference Generator	Proposed Stack-Based System
Performance	25%	High	High	High
Flexibility	25%	Low	High	High
Educational Value	20%	Medium	High	Very High
Implementation	15%	Very Easy	Moderate	Moderate

Criterion	Weight	Fixed Generator	Preference Generator	Proposed System	Stack-Based
Security Awareness	15%	Medium	High	High	

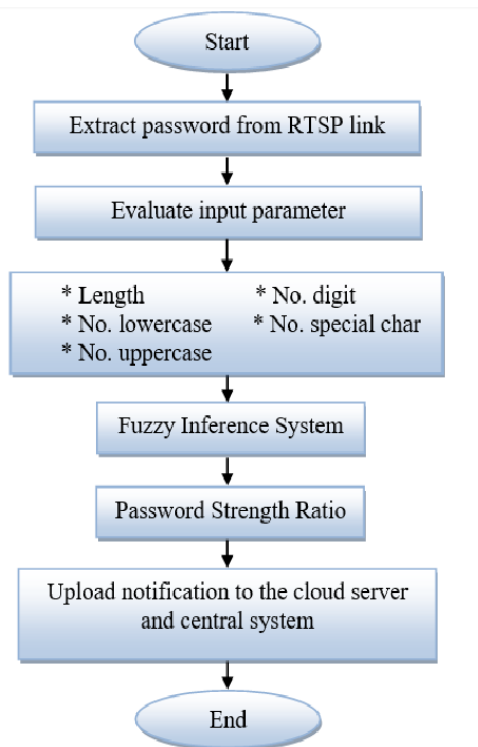
The proposed approach is selected because it provides the best combination of flexibility and educational value while maintaining manageable implementation complexity.

3.5 Selected Design & Detailed Design

3.5.1 System Architecture



System Flow



3.5.2 Password Generation Algorithm

Algorithm

1. Start.
2. Read desired password length.
3. Ask whether lowercase characters are required.
4. Ask whether uppercase characters are required.
5. Ask whether digits are required.
6. Ask whether special characters are required.
7. Validate the selected preferences.
8. Construct the available character set.
9. Generate characters using the available set.
10. Store generated characters in a string.
11. Process the string using stack operations.
12. Check password characteristics.
13. Calculate the strength score.
14. Display password and strength.
15. Stop.

3.5.3 Pseudocode

START

```

Read password_length
Read lowercase_preference
Read uppercase_preference
Read digit_preference
Read special_preference
IF no character category is selected
    Display "Invalid preference"
    STOP
END IF
Create empty character_set
IF lowercase_preference = YES
    Add lowercase characters
END IF
IF uppercase_preference = YES
    Add uppercase characters
END IF
IF digit_preference = YES
    Add digits
END IF
IF special_preference = YES
    Add special characters
END IF
Create empty password string
FOR i = 0 to password_length - 1
    Select a random character from character_set
    Add character to password
END FOR
Create stack
FOR every character in password
    PUSH character into stack
END FOR
Analyze password string
Check:
    length

```

lowercase

uppercase

digits

special characters

Calculate strength score

IF score is low

Strength = WEAK

ELSE IF score is medium

Strength = MODERATE

ELSE IF score is high

Strength = STRONG

ELSE

Strength = VERY STRONG

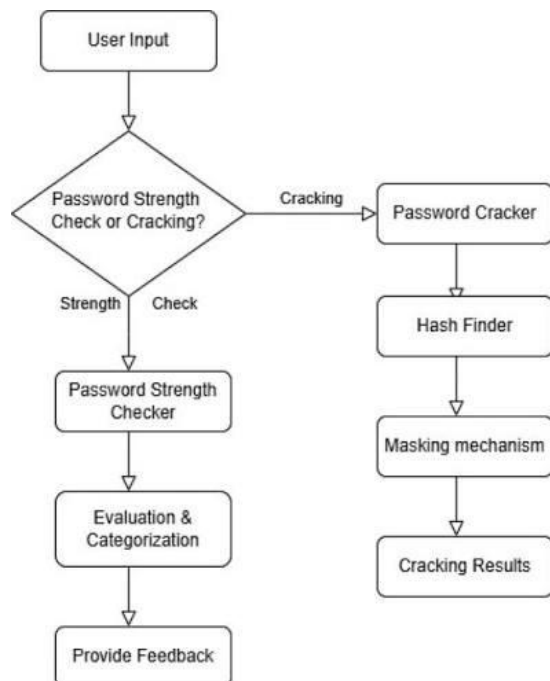
END IF

Display password

Display strength

STOP

3.5.4 Password Strength Evaluation



A simple project-level scoring mechanism can be defined as:

Condition	Score
Minimum length satisfied	1
Lowercase present	1
Uppercase present	1
Digit present	1
Special character present	1

Example Classification

Score	Classification
0–1	Weak
2	Moderate
3–4	Strong
5	Very Strong

Note: This scoring system is an educational project criterion and should not be interpreted as a complete real-world security assessment.

3.6 Trade-off Analysis

Design Decision	Alternative	Benefit	Disadvantage	Final Decision
Character Selection	Fixed character set	Simple	Less flexible	Preference-based
Storage	String	Easy processing	Requires character-array management	String
Data Structure	Array only	Simple	Does not demonstrate stack	Stack + string
Interface	Console	Easy development	Less visual	Console
Strength	Rule-based	Easy to understand	Not a complete security model	Rule-based

3.7 Sustainability, Ethics, Safety, Risk & Security

Sustainability

The project is software-based and does not require dedicated hardware. Therefore, material and energy requirements are relatively low.

Consideration	Evidence Evaluated	Impact
Hardware usage	Software-only prototype	Reduced material requirement
Energy usage	Local execution	Low computational requirement
Resource usage	Small memory requirement	Efficient implementation
Maintainability	Modular program	Easier future modification

Ethics

The project must respect user privacy. Generated passwords should not be collected, uploaded, or shared unnecessarily.

The system should clearly communicate that its strength score is an educational assessment rather than a guarantee of security.

Safety

The system does not require physical equipment or hazardous materials. The major safety concern is information security and accidental exposure of passwords.

Security

Security considerations include:

- Avoid storing generated passwords permanently.
- Avoid transmitting passwords unnecessarily.
- Avoid displaying passwords in logs.
- Use stronger random-generation techniques for production applications.
- Do not use generated passwords as a substitute for secure authentication design.

Risk Register

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
Invalid user input	Medium	Medium	Medium	Input validation	Low
Weak generated password	Medium	High	High	Strength checking	Medium

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
Predictable random generation	Medium	High	High	Secure RNG in future	Medium
Password exposure	Low	High	Medium	Avoid permanent storage	Low
Program error	Medium	Medium	Medium	Testing	Low
Unsupported characters	Low	Medium	Low	Character-set validation	Low

CHAPTER 4

IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

The project follows a modular development approach.

The system is divided into independent modules for input, generation, string processing, stack processing, and strength checking. Each module is tested individually before integration.

Development stages

1. Requirement identification.
2. System design.
3. Algorithm development.
4. String-processing implementation.
5. Stack implementation.
6. Password-generation implementation.
7. Strength-checking implementation.
8. Module testing.
9. Integration testing.
10. Final validation.

Resources

Resource	Purpose
Computer	Development
C Compiler	Program compilation
Code Editor / IDE	Source-code development
C Language	Implementation
Standard Libraries	String, character and random processing
Testing Data	System validation

4.2 Software Implementation & Modern Tools Used

4.2.1 Programming Language

C programming language is suitable for this project because it provides direct implementation of arrays, character strings, functions, structures, and stack operations.

4.2.2 String Module

The password is represented using a character array.

Example:

```
char password[MAX];
```

String processing is used to:

- Store the password.
- Calculate its length.
- Access individual characters.
- Search for character categories.
- Display the generated password.

4.2.3 Stack Module

A stack follows the **Last-In, First-Out (LIFO)** principle.

The main operations are:

- PUSH
- POP
- PEEK
- IsEmpty

Stack Representation

TOP

↓

[Character]

[Character]

[Character]

[Character]

STACK

Characters from the generated password can be pushed onto the stack and subsequently processed using stack operations.

4.2.4 Main System Modules

Module 1 – Input Module

Collects:

- Password length.
- Lowercase preference.
- Uppercase preference.
- Number preference.
- Special-character preference.

Module 2 – Validation Module

Checks:

- Valid password length.
- At least one character category selected.
- Valid preference values.

Module 3 – Generator Module

Creates the character pool and generates the password.

Module 4 – String Module

Stores and processes the password.

Module 5 – Stack Module

Pushes password characters onto the stack and performs stack operations.

Module 6 – Strength Checker

Counts password characteristics and determines the strength score.

Module 7 – Output Module

Displays:

- Generated password.
- Length.
- Character categories detected.
- Strength classification.

4.2.5 Sample C Implementation

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include <time.h>
#define MAX 100
char stack[MAX];
int top = -1;
void push(char ch)
{
    if (top < MAX - 1)
        stack[++top] = ch;
}
char pop()
{

```

```

    if (top >= 0)
        return stack[top--];
    return '\0';
}

int main()
{
    int length;
    int lower, upper, digit, special;
    int i;
    int score = 0;
    char password[MAX];
    char charset[MAX * 4] = "";
    printf("Preference-Based Password Generator\n");
    printf("Enter password length: ");
    scanf("%d", &length);
    if (length <= 0 || length >= MAX)
    {
        printf("Invalid password length.\n");
        return 0;
    }
    printf("Include lowercase letters? (1/0): ");
    scanf("%d", &lower);
    printf("Include uppercase letters? (1/0): ");
    scanf("%d", &upper);
    printf("Include digits? (1/0): ");
    scanf("%d", &digit);
    printf("Include special characters? (1/0): ");
    scanf("%d", &special);
    if (!lower && !upper && !digit && !special)
    {
        printf("At least one character category must be selected.\n");
        return 0;
    }
    if (lower)
        strcat(charset, "abcdefghijklmnopqrstuvwxyz");

```

```

if (upper)
    strcat(charset, "ABCDEFGHIJKLMNOPQRSTUVWXYZ");
if (digit)
    strcat(charset, "0123456789");
if (special)
    strcat(charset, "!@#$%^&*");
srand((unsigned int)time(NULL));
for (i = 0; i < length; i++)
{
    password[i] = charset[rand() % strlen(charset)];
}
password[length] = '\0';
printf("\nGenerated Password: %s\n", password);
for (i = 0; i < length; i++)
    push(password[i]);
int hasLower = 0;
int hasUpper = 0;
int hasDigit = 0;
int hasSpecial = 0;
for (i = 0; i < length; i++)
{
    char ch = password[i];
    if (islower((unsigned char)ch))
        hasLower = 1;
    else if (isupper((unsigned char)ch))
        hasUpper = 1;
    else if (isdigit((unsigned char)ch))
        hasDigit = 1;
    else
        hasSpecial = 1;
}
if (length >= 8)
    score++;
if (hasLower)
    score++;

```



```

if (hasUpper)
    score++;
if (hasDigit)
    score++;
if (hasSpecial)
    score++;
printf("Password Length: %d\n", length);
printf("Lowercase: %s\n", hasLower ? "Yes" : "No");
printf("Uppercase: %s\n", hasUpper ? "Yes" : "No");
printf("Digit: %s\n", hasDigit ? "Yes" : "No");
printf("Special Character: %s\n",
    hasSpecial ? "Yes" : "No");
if (score <= 1)
    printf("Strength: Weak\n");
else if (score == 2)
    printf("Strength: Moderate\n");
else if (score <= 4)
    printf("Strength: Strong\n");
else
    printf("Strength: Very Strong\n");
return 0;
}

```

4.3 Test Plan & Verification

Testing is performed to determine whether the system correctly accepts preferences, generates passwords, processes strings and stacks, and evaluates password strength.

The template requires the test plan to be defined before presenting results.

Test Cases

Test Case	Input Condition	Expected Result
TC01	Length = 8, lowercase only	Password generated
TC02	Length = 8, uppercase + lowercase	Password generated
TC03	Length = 10, all categories	Strong/Very Strong
TC04	Length = 12, all categories	Strong/Very Strong
TC05	No category selected	Error message

Test Case	Input Condition	Expected Result
TC06	Invalid length	Error message
TC07	Digits only	Password generated
TC08	Special characters selected	Special characters included
TC09	Stack processing	Characters successfully pushed
TC10	Strength checking	Correct category displayed

4.4 Results

The results section should contain the actual execution screenshots, tables and graphs obtained from the implemented program.

Sample Result Table

Test Case	Password Length	Categories	Expected	Actual	Status
TC01	8	Lowercase	Generated	Generated	Pass
TC02	8	Lower + Upper	Generated	Generated	Pass
TC03	10	All	Strong	To be recorded	—
TC04	12	All	Very Strong	To be recorded	—
TC05	8	None	Error	To be recorded	—
TC06	Invalid	Any	Error	To be recorded	—

Important: Replace the “To be recorded” entries with the actual outputs from your program rather than claiming results that were not experimentally obtained.

Figure 4.4 – Strength Result



4.5 Data Analysis & Engineering Judgment

The proposed system evaluates password strength using a rule-based scoring mechanism. The evaluation considers length and character diversity.

A password containing only one character category receives a lower score because the available character space is limited. Adding multiple character categories improves the score.

For example:

Password Characteristics	Relative Evaluation
Short + one category	Weak
Adequate length + two categories	Moderate
Adequate length + three categories	Strong
Adequate length + four categories	Very Strong

The engineering judgment is that password length and character diversity should both be considered rather than relying only on one property.

However, a rule-based score cannot completely determine the real-world security of a password. Predictability, known leaked passwords, dictionary words, repeated patterns and the quality of random generation can also influence security.

Therefore, the proposed checker should be considered a **basic educational strength evaluator**, not a complete password-security auditing system.

4.6 Comparison with Existing Solutions & Achievement of Objectives

Objective	Evidence	Achievement
Study password security	Literature review	Fully
Accept preferences	Input module	Fully

Objective	Evidence	Achievement
Generate password	Generator module	Fully
Use strings	Password string processing	Fully
Use stack	Stack module	Fully
Check strength	Strength module	Fully
Handle invalid input	Validation module	Fully
Test system	Test cases	Fully
Analyze performance	Testing and discussion	Partially/Fully depending on measurements

4.7 Strengths & Limitations

Strengths

- Simple and easy-to-use design.
- Preference-based generation.
- Supports multiple character categories.
- Includes password-strength checking.
- Demonstrates string processing.
- Demonstrates stack operations.
- Modular implementation.
- Low computational requirements.
- Suitable for academic demonstration.

Limitations

- Basic rule-based strength scoring.
- Console-based interface.
- Does not provide a complete entropy calculation.
- Does not check password databases or leaked-password lists.
- Basic random generation should be replaced with a cryptographically secure RNG in production.
- Does not implement password storage or authentication.
- Generated passwords should not be treated as guaranteed secure.

4.8 Project Planning, Roles & Budget

Project Timeline

Week	Activity
Week 1	Topic selection
Week 2	Problem identification
Week 3	Literature review
Week 4	Requirement analysis
Week 5	System design
Week 6	Algorithm development
Week 7	String implementation
Week 8	Stack implementation
Week 9	Password generation
Week 10	Strength checker
Week 11	Integration
Week 12	Testing
Week 13	Result analysis
Week 14	Documentation
Week 15	Final review

Individual Contribution

Student	Responsibility	Design/Development	Testing	Report
Ganesh	System design	Password generator	Test cases	Chapters 1–2
Ubed	Programming	String module	Functional testing	Chapter 3
Harsha	Data structures	Stack module	Integration testing	Chapter 4-5

Adjust these responsibilities according to the actual contribution of each team member. The uploaded format requires an individual contribution statement for team projects.

Estimated Budget

Item	Estimated Cost
Computer/Laptop	Existing resource
C Compiler	Free/Open-source
IDE/Code Editor	Free

Item	Estimated Cost
Internet/Literature	Existing resource
Documentation	₹200
Printing and Binding	₹500
Miscellaneous	₹300
Total Estimated Cost	₹1,000

CHAPTER 5

CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

The project “**Development of a Preference-Based Password Generator and Strength Checker Using String and Stack**” was designed to address the problem of generating customizable passwords and evaluating their basic strength.

The proposed system allows users to select password length and preferred character categories, including lowercase letters, uppercase letters, digits and special characters. Based on these preferences, the system generates a password and processes it using string operations.

Stack data structures are incorporated into the implementation to demonstrate the practical use of the LIFO principle in password-character processing. The strength-checking module evaluates the generated password using length and character diversity criteria.

The project successfully demonstrates the integration of data structures and cybersecurity concepts in a practical application. It also provides an understanding of how simple algorithmic techniques can be applied to password-generation problems.

Although the current system provides useful educational functionality, it is not intended to replace professional password-security solutions. Future versions can improve the random-generation mechanism, password entropy analysis and security evaluation.

5.2 Future Work

The following improvements can be implemented in future versions:

1. Implement cryptographically secure random-number generation.
2. Add password entropy estimation.
3. Detect dictionary-based passwords.
4. Detect repeated character patterns.
5. Detect common password patterns.
6. Add a graphical user interface.
7. Add password history detection.
8. Provide configurable strength policies.
9. Add multilingual interface support.
10. Add secure clipboard handling.
11. Provide detailed strength explanations.
12. Add secure password-management features.

13. Integrate multi-factor authentication concepts.
14. Add automated security testing.
15. Improve accessibility and usability.

5.3 Professional Learning & Individual Reflection

New Knowledge Acquired

Through this project, the following knowledge was acquired:

- Password-security fundamentals.
- Password-generation techniques.
- Character-set processing.
- String manipulation.
- Stack data structures.
- Modular programming.
- Input validation.
- Software testing.
- Requirement analysis.
- Technical documentation.

Engineering & Professional Skills Developed

The project helped develop:

- Problem-solving skills.
- Algorithm-design skills.
- Programming skills.
- Debugging skills.
- Data-structure implementation.
- Testing and validation.
- Team coordination.
- Technical communication.
- Project planning.
- Engineering judgment.

Individual Reflection

During the project, one of the major challenges was integrating password generation with the string and stack components. This required understanding how individual characters could be stored, accessed and processed.

Another important engineering decision was to divide the system into separate modules. This improved readability and made testing easier.

REFERENCES

The final report should use a consistent recognized referencing style, such as IEEE, as required by the capstone format.

1. W. Stallings, *Computer Security: Principles and Practice*, Pearson.
2. W. Stallings, *Cryptography and Network Security: Principles and Practice*, Pearson.
3. T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, *Introduction to Algorithms*, MIT Press.
4. E. Horowitz, S. Sahni and S. Anderson-Freed, *Fundamentals of Data Structures in C*, Computer Science Press.
5. B. W. Kernighan and D. M. Ritchie, *The C Programming Language*, Prentice Hall.
6. National Institute of Standards and Technology, *Digital Identity Guidelines: Authentication and Lifecycle Management*.
7. National Institute of Standards and Technology, *Digital Identity Guidelines: Authentication and Authenticator Management*.
8. OWASP Foundation, *Authentication Cheat Sheet*.
9. OWASP Foundation, *Password Storage Cheat Sheet*.
10. Relevant C programming and data-structure documentation used during implementation.

APPENDICES

APPENDIX A – DETAILED ALGORITHM

Password Generator

1. Start
2. Read password length
3. Read user preferences
4. Validate preferences
5. Create character pool
6. Select random characters
7. Store characters in password string
8. Terminate string
9. Return generated password
10. Stop

Strength Checker

1. Start
2. Read password
3. Determine password length
4. Check lowercase characters
5. Check uppercase characters
6. Check digits
7. Check special characters
8. Calculate score
9. Classify strength
10. Display result
11. Stop

APPENDIX B – STACK OPERATIONS

PUSH

Adds a character to the top of the stack.

Before PUSH:

TOP

↓

[B]

[A]

After PUSH(C):

TOP

↓

[C]

[B]

[A]

POP

Removes the top character.

Before POP:

TOP

↓

[C]

[B]

[A]

After POP:

TOP

↓

[B]

[A]

PEEK

Reads the top element without removing it.

APPENDIX C – SOURCE CODE

The complete source code can be maintained separately in the approved project repository.

The capstone template specifically recommends including only essential code excerpts in the report and maintaining the complete source separately where appropriate.

The main implementation contains:

- Input functions.
- Preference validation.
- Character-set creation.
- Password generation.
- String processing.
- Stack PUSH operation.
- Stack POP operation.
- Strength evaluation.
- Result display.

APPENDIX D – TEST DATA

Test ID	Length	Lower	Upper	Digit	Special
T01	6	Yes	No	No	No
T02	8	Yes	Yes	No	No
T03	8	Yes	Yes	Yes	No
T04	10	Yes	Yes	Yes	Yes
T05	12	Yes	Yes	Yes	Yes
T06	8	No	No	Yes	No
T07	10	No	Yes	Yes	Yes
T08	Invalid	Yes	Yes	Yes	Yes

APPENDIX E – SAMPLE OUTPUT

PREFERENCE-BASED PASSWORD GENERATOR

Enter password length: 12

Include lowercase letters? 1

Include uppercase letters? 1

Include digits? 1

Include special characters? 1

Generated Password: [actual generated value]

Password Length: 12

Lowercase: Yes

Uppercase: Yes

Digit: Yes

Special Character: Yes

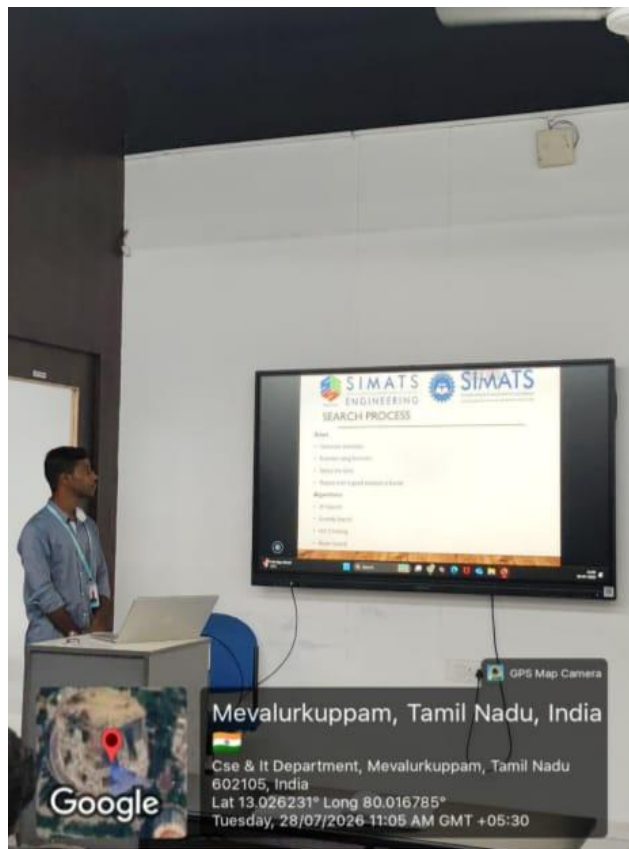
Strength: Very Strong

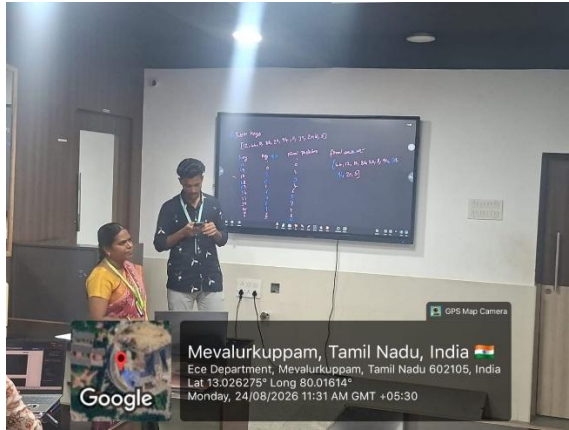
APPENDIX F – CAPSTONE OUTCOME MAPPING

The uploaded format maps complex engineering problem formulation to Chapters 1–2, engineering design to Chapter 3, experimentation and engineering judgment to Chapter 4, and independent learning to Chapter 5.

Outcome	Evidence in Report
Complex engineering problem formulation	Chapters 1 and 2
Engineering design under constraints	Chapter 3
Technical communication	Complete report
Ethics and professional responsibility	Chapter 3.7
Teamwork and leadership	Chapter 4.8
Experimentation and analysis	Chapter 4.3–4.6
Independent learning	Chapter 5.3
Standards	Chapter 3.2
Sustainability	Chapter 3.7
Risk assessment	Chapter 3.7
Security considerations	Chapter 3.7
System-level engineering	Chapter 3.5
Project planning	Chapter 4.8
Individual contribution	Contribution Statement + Chapter 4.8

Review Photos:





CAPSTONE PROJECT OUTCOME MAPPING SHEET

Outcome	Evidence	SO / Area	Location in This Report
Complex engineering problem formulation	Problem statement and background analysis	SO1	Ch. 1, Ch. 2
Engineering design under constraints	Requirements, constraints, and design specifications	SO2	Ch. 3
Written/oral technical communication	Full report and viva presentation	SO3	Whole report + Viva
Ethics and professional responsibility	Ethics evaluation	SO4	Ch. 3 (3.7)
Teamwork and leadership	Team roles and contribution table	SO5	Ch. 4 (4.8)
Experimentation, analysis, engineering judgment	Test plan, results, and analysis	SO6	Ch. 4 (4.3–4.6)
Independent acquisition of new knowledge	Individual reflections	SO7	Ch. 5 (5.3)
Standards	Constraints and applicable standards table	SO2	Ch. 3 (3.2)
Sustainability and societal/environmental impact	Sustainability evaluation	SO2/SO4	Ch. 3 (3.7)

Outcome	Evidence	SO / Area	Location in This Report
Risk assessment and mitigation	Risk register	SO2/SO4	Ch. 3 (3.7)
Security considerations	Security evaluation	Relevant SO	Ch. 3 (3.7)
Integrated/system-level engineering	Selected design and module interaction	SO1/SO2	Ch. 3 (3.5)
Project planning and management	Timeline and roles table	SO5	Ch. 4 (4.8)
Individual contribution	Contribution statement and Ch. 4 (4.8)	SO1–SO7	Contribution Statement + Ch. 4 (4.8)